

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-19 Créditos: 3	
---	--	---

Taller 3: Clasificación del Mundial 2026 con Lógica

Ustedes son ejecutivos de la FIFA que se preparan para el próximo Mundial. Los medios y los aficionados necesitan saber, partido a partido, quién ya clasificó o quedó eliminado aunque aún queden partidos por jugar. Su misión: construir un sistema que modele reglas formales con lógica y produzca esta información en tiempo real.

Para ello aplicarán lógica proposicional (model checking, forma normal conjuntiva) y lógica de predicados vía escenarios posibles: a partir de resultados conocidos se enumeran todos los desenlaces de los partidos pendientes y se deduce solo lo que vale en todos los mundos. Su trabajo es implementar las funciones clave del motor proposicional y los predicados de clasificación.

Objetivos de aprendizaje

- Implementar funciones de model checking para lógica proposicional.
- Implementar las transformaciones necesarias para convertir fórmulas a Forma Normal Conjuntiva (CNF).
- Modelar las reglas de la fase de grupos del Mundial 2026 con predicados evaluados sobre escenarios posibles (*entailment* en todos los mundos).
- Comparar y reflexionar sobre los diferentes métodos de inferencia lógica.

Recursos de Apoyo

El proyecto incluye varios recursos para ayudar a entender el motor de lógica. Estos recursos no son evaluados. La calificación se basa exclusivamente en las pruebas unitarias y las preguntas de análisis.

- Guía de Objetos en Python: `notebooks/guia_objetos_python.ipynb`
- Guía interactiva sobre Model Checking: `notebooks/parte1_model_checking.ipynb`
- Guía interactiva sobre transformación a CNF: `notebooks/parte2_cnf.ipynb`
- Guía interactiva sobre lógica de predicados: `notebooks/parte3_predicados.ipynb`
- Interfaz de terminal para explorar la fase de grupos de forma interactiva: `uv run main.py`
- Estructura del repositorio e instrucciones de inicio rápido: `README.md`

Punto 1: Model Checking (20%)

Implementen las 5 funciones en `src/model_checking.py`. Cada función tiene un *docstring* con su descripción, ejemplo de uso y *hints* para la implementación.

Funciones a implementar

- `get_all_models(atoms)`: Genera todos los modelos posibles (asignaciones de verdad) para un conjunto de átomos. Para n átomos, debe generar 2^n modelos.
- `check_satisfiable(formula)`: Determina si una fórmula es satisfacible. Retorna (True, modelo) si encuentra un modelo que satisface la fórmula, o (False, None) si es insatisfacible.
- `check_valid(formula)`: Determina si una fórmula es una tautología (válida en todo modelo posible).
- `check_entailment(kb, query)`: Determina si $KB \models query$, es decir, si la base de conocimiento implica la consulta.
- `truth_table(formula)`: Genera la tabla de verdad completa de una fórmula.

Punto 2: Conversión a CNF (40%)

Implementen las 5 funciones de transformación en `src/cnf_transform.py`. La función `eliminate_double_negation` y el pipeline completo `to_cnf` ya están dados como referencia.

Funciones a implementar

- `eliminate_iff(formula)`: Elimina bicondicionales $A \leftrightarrow B$
- `eliminate_implication(formula)`: Elimina implicaciones $A \rightarrow B$
- `push_negation_inward(formula)`: Aplica las leyes de De Morgan.
- `distribute_or_over_and(formula)`: Distribuye Or sobre And.
- `flatten(formula)`: Aplana conjunciones y disyunciones anidadas.

Punto 3: Lógica de Predicados (30%)

En esta sección modelarán la clasificación de la fase de grupos con funciones predicado evaluadas sobre mundos posibles. El motor de escenarios ya está implementado en `src/logic/predicate/engine.py`: dado un estado parcial (resultados con goles conocidos + partidos pendientes), genera los 3^k escenarios posibles y permite preguntar si una propiedad vale en todos. Esto permite definir si un equipo tiene garantizado clasificar o ser eliminado del torneo.

3a. Implementación de predicados (15%)

Implementen predicados de ranking y de clasificación/eliminación en `src/worldcup/rules.py`:

- `points(world, team)`: Cálculo de los puntos de un equipo en un escenario particular.
- `goal_difference(world, team)`: Diferencia de goles en dicho escenario.
- `teams_below(world, team)`: Cuántos rivales quedan estrictamente debajo en ranking.
- `teams_above(world, team)`: Cuántos rivales quedan estrictamente arriba en ranking.
- `definitely_qualified(engine, state, team)`: Evalúa si en todo escenario al menos 2 rivales quedan debajo (clasificación matemática top-2).
- `definitely_eliminated(engine, state, team)`: Evalúa si en todo escenario al menos 2 rivales quedan arriba.

3b. Preguntas de análisis cualitativo (15%)

Respondan las siguientes preguntas a partir del modelado de la fase de grupos del mundial en lógica de predicados y su experimentación con los motores de lógica proporcionados:

- ¿Qué significa que una conclusión se derive con `holds_in_all`? ¿En qué se parece y en qué se diferencia de $KB \models \phi$ en model checking proposicional?
- ¿Qué es fijo y qué varía entre los escenarios de mundos posibles? ¿Qué se perdería si solo usáramos constantes sin cuantificar sobre escenarios?
- ¿Por qué los partidos pendientes usan marcadores 1-0 / 0-0 / 0-1? ¿Qué se gana y qué se pierde con esa simplificación?
- ¿Por qué a menudo no se puede determinar si un equipo clasifica hasta la jornada 3 aunque la tabla ya sugiera un orden? ¿Qué condición es necesaria para poder asegurar clasificación?
- Nótese que este taller no modela la regla de los mejores terceros. ¿Qué haría falta para modelarlos con el mismo estilo de escenarios?

Punto 4: Reflexión (10%)

Escriban una reflexión que aborde los siguientes puntos:

- (5%) Para cada uno de los puntos anteriores, expliquen brevemente: ¿Qué enfoque utilizaron para su implementación? ¿Qué dificultades encontraron? ¿Cómo verificaron que su implementación era correcta?
- (5%) Escriban una reflexión concisa sobre la co-construcción de la solución con apoyo de la IA (vea política de uso más adelante), indicando qué aprendizajes obtuvieron, si las correcciones de la IA fueron mejoras secundarias, y los beneficios y limitaciones de la IA en este contexto.

Evaluación

Este taller incluye un conjunto de pruebas unitarias en `tests/` que pueden ser utilizadas para validar su implementación. Pueden ser ejecutadas con el comando: `uv run pytest -v`. Se debe tener en cuenta que la evaluación final se realizará con un conjunto distinto de pruebas, por lo cual el desempeño en las pruebas suministradas no puede considerarse equivalente a la nota obtenida en el taller. Se recomienda que utilicen dichas pruebas para encontrar errores durante el desarrollo, e incluso que añadan pruebas propias para asegurarse de que sus algoritmos manejan casos de frontera de forma adecuada.

Política de Uso de IA Generativa y presentación en el trabajo a entregar

Para este taller se espera que ustedes desarrollen una primera versión de la solución de forma completamente autónoma, usando su propio criterio, conocimiento y creatividad antes de recurrir a herramientas de IA. La IA puede emplearse después para mejoras puntuales, refactorización, comentarios de calidad o apoyo en la corrección de errores, pero nunca como sustituto del esfuerzo personal ni como generador principal del código. Usen estas herramientas con criterio profesional, asumiendo responsabilidad sobre su proceso de aprendizaje y entendiendo que lo que más valor tiene aquí es el camino que ustedes recorren para llegar a la solución, no solo el resultado final.

Todos los prompts y las versiones de código generadas con apoyo de IA deben quedar registrados dentro de los archivos del proyecto. Si utilizan IA para refactorizar u optimizar su solución, incluya en el archivo: (1) la versión inicial del código, (2) los prompts que utilizó para refinarla y (3) la versión final del código. Al finalizar, deje como código activo únicamente la versión final y conserve la versión inicial y todos los prompts en forma de comentarios dentro del mismo archivo.

Entrega del trabajo

- Guarden todo su trabajo (incluyendo el documento PDF) en una carpeta comprimida (.zip).
- Suban el trabajo al espacio correspondiente en Bloque Neón a más tardar la fecha y hora indicadas (un solo miembro del grupo debe subir el trabajo).
- Recuerden incluir los nombres y códigos de todos los integrantes de su grupo.

Nota: Al enviar su solución, ustedes declaran que el código entregado en la primera versión de cada punto es de su autoría y que las versiones que utilizan IA fueron la respuesta a los prompts incluidos, que comprenden el funcionamiento en todas las versiones y que aceptan que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.